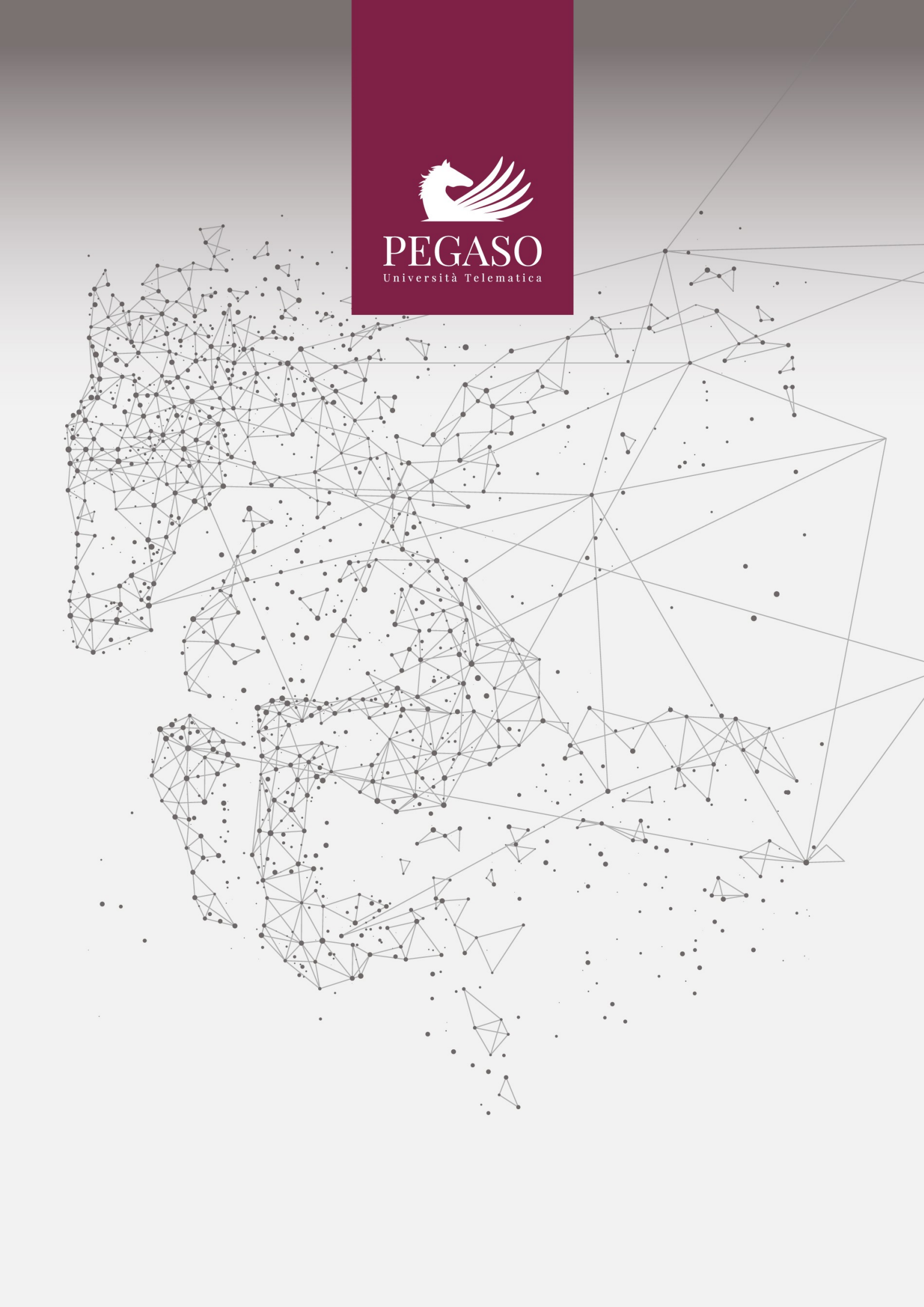

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. INTRODUZIONE ALLE ESPRESSIONI REGOLARI 4
3. RICERCHE E PATTERN TRAMITE ESPRESSIONI REGOLARI 7
4. PYTHON E LE ESPRESSIONI REGOLARI 10
BIBLIOGRAFIA 18

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono le espressioni regolari, modelli che permettono di verificare la corrispondenza letterale del testo, ripetizioni, composizioni di modelli, diramazioni ed altre regole sofisticate. Vengono brevemente introdotti dal punto di vista teorico le ricerche a pattern tramite espressioni regolari e si introducono gli strumenti che Python mette a disposizione per le espressioni regolari. In particolare, vengono illustrate le funzioni split, match e search del modulo re.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Introduzione alle espressioni regolari

Le **espressioni regolari** sono dei **modelli**, descritti con una sintassi formale, per cercare corrispondenze in un testo. I modelli sono interpretati come un **insieme di istruzioni**, le quali sono poi eseguite con una stringa come input per produrre un sottoinsieme di corrispondenze oppure una versione modificata della stringa originale.

Il termine "espressione regolare" (regular expression) viene spesso abbreviato come "**regex**".

Le espressioni possono comprendere **corrispondenza letterale del testo, ripetizione, composizione di modello, diramazione** ed altre sofisticate regole.

Un grande numero di problemi di elaborazione di testo è più semplice da risolvere usando una espressione regolare al posto di creare un analizzatore ed elaboratore lessicale per lo scopo specifico. Le espressioni regolari sono tipicamente usate in applicazioni che comprendono molta elaborazione di testo. Ad esempio, sono comunemente usate come **modelli di ricerca** in programmi per la **modifica di testo** (text editors) usati dagli sviluppatori.

In merito alla programmazione, le espressioni regolari sono essenzialmente un **piccolo ed altamente specializzato linguaggio di programmazione incorporato in Python** e reso disponibile attraverso il **modulo re**. Utilizzando questo piccolo linguaggio, si specificano le regole per l'insieme di possibili stringhe di cui si vogliono trovare le corrispondenze. Tale insieme potrebbe contenere frasi in inglese, indirizzi di posta elettronica, ecc.

È possibile porre domande tipo **''Questa stringa ha corrispondenza in questo modello?**'' o **''Esiste una corrispondenza con questo modello in un punto della stringa?**''

Il primo **metacarattere** inerente le espressioni regolari è l'asterisco *.

L'asterisco * non cerca il carattere "*", ma specifica che il carattere che lo precede può presentarsi zero o più volte e non esclusivamente una sola volta. Per esempio, ca*t troverà corrispondenza in:

- "ct" (0 caratteri "a");
- "cat" (1 "a");
- "caaat" (3 caratteri "a") e così via.

Altri metacaratteri molto utilizzati sono le **parentesi quadre.** Consideriamo l'espressione "a[bcd]*b". Questa espressione cerca una stringa che inizia con la lettera "a", prosegue con zero o più lettere a scelta tra b,c,d e termina con "b".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Immaginiamo adesso di provare questa espressione regolare con la stringa "abcbd".

Passo	Modello	Spiegazione
1	a	La a corrisponde nell'espressione regolare
2	abcbd	Il motore cerca [bcd]* e va avanti il più possibile, cioè fino alla fine della stringa
3	*Fallimento*	Il motore cerca la b, ma la posizione corrente è alla fine della stringa e quindi fallisce
4	abcb	Torna indietro, adesso [bcd]* corrisponde ad un carattere in meno
5	*Fallimento*	Cerca di nuovo b, ma nella posizione corrente trova il carattere "d"
6	abc	Torna indietro, in questo momento [bcd]* corrisponde solamente con "bc"
7	abcb	Cerca di nuovo b, ma stavolta il carattere nella posizione corrente è "b", la ricerca ha successo

Il lavoro dell'espressione regolare è terminato e corrisponde con "abcb". L'esempio visto mostra come opera l'algoritmo di ricerca, che prima va avanti il più possibile e poi, se non trova la giusta corrispondenza, torna progressivamente indietro riprovando più e più volte.

Un altro metacarattere per le ripetizioni è **+**. Tale operatore evidenzia una o più occorrenze.

È necessario fare attenzione alla differenza tra * e +. L'asterisco *, infatti, cerca zero o più occorrenze, quindi, tutto quello che può essere ripetuto può anche essere del tutto assente. Con + è necessaria, invece, almeno una occorrenza.

Per usare un esempio simile al precedente, ca+t troverà "cat" (1 "a") e "caaat" (3 "a"), ma non troverà "ct".

Esistono altri due qualificatori di ripetizioni. Il carattere punto interrogativo **? cerca una o zero ripetizioni**. Esso è come un marcatore per qualcosa che è facoltativo. Ad esempio, home-?brew troverà sia "homebrew" che "home-brew".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il qualificatore di ripetizioni più complesso è **{m,n}**, dove m e n sono numeri decimali interi.

Ciò significa che devono esserci **almeno m ripetizioni e non più di n.**

Ad esempio, a/{1,3}b corrisponderà con "a/b", "a//b" e "a///b", però non troverà "ab", che non ha barre, o "a////b", che ne ha quattro. Con il quantificatore {m,n} è possibile omettere m o n.

L'omissione di m {,n} viene interpretata come un limite inferiore posto a 0, mentre l'omissione di n {m,} mette il limite superiore a infinito, o meglio, al limite di 2 miliardi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Ricerche e pattern tramite espressioni regolari

Il linguaggio delle espressioni regolari può essere riassunto (parzialmente) attraverso le seguenti tabelle.

Gli operatori di base sono:

Simbolo	Significato
.	Qualsiasi carattere tranne fine riga (newline)
a	Il carattere a
ab	La stringa ab
x|y	x oppure y
\y	Escape di un carattere speciale y, come ^+{}$()[]|\-?.*

Le classi di caratteri sono:

Simbolo	Significato
[a-d]	Uno dei caratteri a, b, c, d
[^a-d]	Un carattere tranne a, b, c, d
\d	Una cifra
\D	Una non cifra
\s	Uno spazio
\S	Un non spazio
\w	Un carattere alfanumerico
\W	Un carattere non-alfanumerico

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I quantificatori sono:

Simbolo	Significato
x*	Zero o più ripetizioni di x
x+	Una o più ripetizioni di x
x?	Zero o una ripetizione di x
x{2}	Esattamente due ripetizioni di x
x{2,5}	Fra due e cinque ripetizioni di x

I caratteri soggetti a escape sono:

Simbolo	Significato
\n	New line
\r	Carriage return
\t	tabulazione

I caratteri che identificano posizioni sono:

Simbolo	Significato
^	Inizio della stringa
\b	Limite della parola
\B	Non limite della parola
$	Fine della stringa

I gruppi sono:

Simbolo	Significato
(x)	Cattura di un gruppo
(?:x)	Non-cattura di un gruppo

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il simbolo backslash \ è un codice di escape in Python. Per essere usato come normale carattere, deve essere preceduto da un altro backslash \\. Tutto ciò da origine ad espressioni regolari molto complesse con un numero molto alto di backslash.

Per risolvere questo aspetto, Python supporta anche le stringhe grezze in cui i backslash non sono interpretati come caratteri di escape. Per definire una stringa grezza si usa il carattere r (di raw) immediatamente prima dell'apice di apertura. Ad esempio, la stringa con espressione regolare \n in Python è:

```
"\\n"
```

Per evitare l'uso sovrabbondante del backslash si usa la seguente forma:

```
r"\n"
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Python e le espressioni regolari

In Python l'espressione regolare è una **stringa con la descrizione di un pattern.** Il modulo re offre un linguaggio per la descrizione di pattern e una collezione di funzioni per r**icerca, suddivisione e sostituzione di stringhe.**

Per rendere la ricerca a pattern più efficiente è possibile **compilare un'espressione regolare** che si prevede di usare più volte utilizzando la funzione **compile**() seguendo il seguente formato:

```
import re
pattern_compilato = re.compile(pattern, flags=0)
```

La compilazione migliora di molto i tempi di ricerca senza incidere sulla correttezza dell'operazione. È possibile specificare anche dei **flag di controllo** della ricerca del pattern (sia in compilazione sia al momento dell'esecuzione). I flag possono essere combinati specificandoli uno dopo l'altro. I flag più importanti sono riportati nella tabella seguente.

re.I	ignora la distinzione fra caratteri maiuscoli e minuscoli
re.L	interpreta le parole rispetto al locale corrente
re.M	attiva la modalità multiriga e consente di usare gli operatori ^ e $ per individuare rispettivamente l'inizio o la fine di una riga
re.U	interpreta i caratteri come set di caratteri Unicode
re.X	ignora gli spazi bianchi

Il modulo re offre tutte le funzioni necessarie e la maggior parte delle funzioni accetta pattern in due forme:

1. grezzo (seconda riga del codice sorgente che segue);
2. compilato (terza riga del codice sorgente che segue).

```
import re
re.function(raw_pattern, ...)
pattern_compilato.function(...)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione **split**() del modulo re con il seguente formato:

```
split(pattern, stringa, maxsplit=0, flags=0)
```

suddivide una stringa in al massimo **maxsplit sottostringhe** in base al pattern e ne restituisce la lista (se maxsplit=0 vengono restituite tutte le sottostringhe). Tale funzione è utilizzabile anche come strumento di riduzione in **token** per l'analisi delle parole.

Nell'esempio successivo si utilizza \W per indicare un carattere non-alfanumerico (ovviamente come separatore) combinando tutte le non-lettere adiacenti:

```
s = re.split(r"\W+", "Ciao, mondo!")
print(s)
```

Il risultato dell'elaborazione precedente risulta essere:

```
['Ciao', 'mondo', '']
```

Nell'esempio successivo si utilizza il carattere \s che è un carattere di spazio bianco equivalente a [\t\n\r\f\v] e un numero massimo di split pari a 3. Dopo il terzo token, verrà riportata tutta la stringa:

```
import re
stringa = "Oggi è proprio una bella giornata"
ris = re.split('\s+', stringa, 3)
print(ris)
```

Il risultato dell'elaborazione precedente risulta essere:

```
['Oggi', 'è', 'proprio', 'una bella giornata']
```

La funzione **match**() del modulo re con il seguente formato:

```
match(pattern ,string, flags =0)
```

controlla se l'**inizio di una stringa** corrisponde all'espressione regolare. La funzione restituisce l'**oggetto corrispondente** oppure **None** se non vengono trovate corrispondenze. L'oggetto corrispondente, se trovato, ha a disposizione le funzioni **start(), end()** e **group()** che restituiscono l'indice iniziale e finale del frammento e il frammento stesso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel codice sorgente che segue, si riportano esempi di uso della funzione match(). In particolare, nel primo caso si richiede se la stringa inizia con una cifra numerica (la cifra viene trovata ed è pari a 1). Nel secondo caso si chiede se la stringa inizia con una o più cifre numeriche (vengono trovate tre cifre pari a 123). Nel terzo e ultimo caso si richiede se la stringa inizia con una o più cifre numeriche (il metodo restituisce None perché la stringa non inizia con un numero).

```
import re

ris = re.match(r"\d", "123 inizia con un numero")
print(ris)
print(ris.group())

ris2 = re.match(r"\d+", "123 inizia con un numero")
print(ris2)
print(ris2.group())

ris3 = re.match(r"\d+", "Stringa che non inizia con un numero")
print(ris3)
```

Il risultato dell'elaborazione precedente risulta essere:

```
<re.Match object; span=(0, 1), match='1'>
1
<re.Match object; span=(0, 3), match='123'>
123
None
```

Nel caso successivo si verifica che la stringa inizi con la parola "espressioni" senza considerare il maiuscolo/minuscolo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import re
stringa = "Espressioni regolari - programmazione python"
ris = re.match(r".*", stringa, re.I)
print(ris.group())
```

Il risultato dell'elaborazione precedente risulta essere:

```
Espressioni
```

Nel caso successivo si verifica che la stringa inizi con un qualsiasi carattere senza considerare il maiuscolo/minuscolo

Il risultato dell'elaborazione precedente risulta essere:

```
Espressioni regolari - programmazione python
```

Nel caso successivo si verifica che la stringa inizi con un singolo carattere senza considerare il maiuscolo/minuscolo.

```
import re
stringa = "Espressioni regolari - programmazione python"
ris = re.match(r".?", stringa, re.I)
print(ris.group())
```

Il risultato dell'elaborazione precedente risulta essere:

```
E
```

Nel caso successivo si verifica che la stringa inizi con un numero di telefono composto da 3 cifre seguito dal carattere slash e. successivamente, da altre 7 cifre.

```
import re
stringa = "320/1234567 numero di telefono di Paolo Rossi"
ris = re.match(r"\d{3}/\d{7}", stringa, re.I)
print(ris.group())
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione precedente risulta essere:

```
320/1234567
```

Il caso successivo è simile al caso precedente con l'unica differenza che si prevede che tra le 3 cifre del prefisso e le successive 7 cifre del numero di telefono ci possa essere uno dei seguenti caratteri: trattino -, spazio, slash /.

```
import re
stringa = "320-1234567 numero di telefono di Paolo Rossi"
ris = re.match(r"\d{3}[- /]\d{7}", stringa, re.I)
print(ris.group())
```

Il risultato dell'elaborazione precedente risulta essere:

```
320-1234567
```

La funzione **search**() del modulo re con il seguente formato:

```
search(pattern, string, flags = 0)
```

controlla se una **qualsiasi parte di una stringa corrisponde all'espressione regolare**. La funzione restituisce l'oggetto corrispondente o None se non vengono trovate corrispondenze. Il risultato della funzione search() può essere utilizzato anche all'interno di una **struttura decisionale** (trovato/non trovato). La funzione search() si utilizza al posto della funzione match() se il **frammento da individuare non si trova all'inizio della stringa.**

Si riportano due esempi di uso della funzione search() che differiscono solo per l'inclusione (primo caso) e la non inclusione (secondo caso) del flag re.I che indica di ignorare la distinzione minuscole/maiuscole. In particolare, si ricerca in una stringa la prima parola composta da lettere. Nel primo caso, il risultato preleverà l'intera prima parola inclusa la "Q", mentre nel secondo caso prenderà solo la parte con le lettere minuscole.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import re

stringa = "123456 Questa stringa ha almeno una 1223 lettera 1233456"
ris = re.search(r"[a-z]+", stringa , re.I)
print("Risultato della funzione search:\n",ris)
print("Ris.group()=",ris.group())
print()
ris = re.search(r"[a-z]+", stringa)
print("Risultato della funzione search con distinzione \
maiuscole/minuscole:\n",ris)
print("Ris.group()=",ris.group())
```

Il risultato dell'elaborazione precedente risulta essere:

```
Risultato della funzione search:
 <re.Match object; span=(7, 13), match='Questa'>
Ris.group()= Questa

Risultato della funzione search con distinzione maiuscole/minuscole:
 <re.Match object; span=(8, 13), match='uesta'>
Ris.group()= uesta
```

Il codice sorgente che segue mostra il diverso utilizzo delle funzioni match() e search() a parità di parametri passati come argomenti. Si vuole cercare la parola "programmazione" all'interno della stringa. Nel primo caso, la funzione match() non troverà la parola perché "programmazione" non si trova all'inizio della stringa. Nel secondo caso, invece, la parola "programmazione" verrà trovata.

```
import re
stringa = "Corso di programmazione Python"
ris = re.match(r"programmazione", stringa, re.I)
ris2 = re.search(r"programmazione", stringa, re.I)
print("match ris = ",ris)
print("search ris = ",ris2)
print("search ris group() = ",ris2.group())
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione precedente risulta essere:

```
match ris =  None
search ris =  <re.Match object; span=(9, 23), match='programmazione'>
search ris group() =  programmazione
```

O

vviamente, utilizzando i caratteri che esprimono le posizioni come il carattere ^ che indica di ricercare il pattern all'inizio della stringa, la funzione search() si comporta esattamente come la funzione match() come visibile nel codice sorgente che segue:

```
import re
stringa = "Corso di programmazione Python"
ris = re.search(r"^programmazione", stringa, re.I)
ris2 = re.search(r"^corso", stringa, re.I)
print("search programmazione ris = ",ris)
print("search corso ris = ",ris2)
```

Il risultato dell'elaborazione precedente risulta essere:

```
search programmazione ris =  None
search corso ris =  <re.Match object; span=(0, 5), match='Corso'>
```

Per ricercare alla fine della stringa con la funzione search() si dovrà utilizzare il carattere dollaro $ come nei casi seguenti:

```
import re
stringa = "Corso di programmazione Python"
ris = re.search(r"programmazione$", stringa, re.I)
ris2 = re.search(r"python$", stringa, re.I)
print("search programmazione ris = ",ris)
print("search corso ris = ",ris2)
```

Il risultato dell'elaborazione precedente risulta essere:

```
search programmazione ris =  None
search corso ris =  <re.Match object; span=(24, 30), match='Python'>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si riportano, ora, tre esempi di utilizzo della funzione search() in combinazione con uno o più valori di flag. Nel primo caso si ricerca una I maiuscola che non verrà trovata. Nel secondo caso, il flag re.I ci permette di ignorare la differenza maiuscolo/minuscolo, quindi la i verrà trovata. Nel terzo e ultimo caso, vengono utilizzati entrambi i flag re.I e re.M che indicano rispettivamente di ignorare la differenza maiuscolo/minuscolo e di cominciare dall'inizio della linea (utilizzo di ^ all'interno della stringa in cui ricercare).

```
import re
s = re.search("I", "Ciao")
print(s)
#Output: nessuno, lettera i c'è ma minuscola

s = re.search("I", "Ciao", re.I)
print(s)
#Output: i

s = re.search("I", "^Ciao", re.I | re.M)
print(s)
#Output: i, la ricerca sarà effettuata dall'inizio della linea
#senza considerare maiuscolo/minuscolo
```

Il risultato dell'elaborazione precedente risulta essere:

```
None
<re.Match object; span=(1, 2), match='i'>
<re.Match object; span=(2, 3), match='i'>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev, D. (2019). Data Science con Python. Apogeo.
- Kuchling, A.M. (2021). HOWTO sulle Espressioni Regolari

  https://www.python.it/doc/howto/Regex/regex-it/regex-it.html

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).